

A simple and easy-to-use math library for games programming

[raylib Discord server][github.com/raysan5/raylib/raymath][raylib.h]

v2.0 quick reference card

module: raymath

```
// Utils math
float Clamp(float value, float min, float max) // Clamp float value
float Lerp(float start, float end, float amount) // Calculate linear interpolation between two floats
float Normalize(float value, float start, float end) // Normalize input value within input range
float Remap(float value, float inputStart, float inputEnd, float outputStart, float outputEnd) // Remap input value within input range to output range
float Wrap(float value, float min, float max) // Wrap input value from min to max
int FloatEquals(float x, float y) // Check whether two given floats are almost equal

// Vector2 math
Vector2 Vector2Zero(void) // Vector with components value 0.0f
Vector2 Vector2One(void) // Vector with components value 1.0f
Vector2 Vector2Add(Vector2 v1, Vector2 v2) // Add two vectors (v1 + v2)
Vector2 Vector2AddValue(Vector2 v, float add) // Add vector and float value
Vector2 Vector2Subtract(Vector2 v1, Vector2 v2) // Subtract two vectors (v1 - v2)
Vector2 Vector2SubtractValue(Vector2 v, float sub) // Subtract vector by float value
float Vector2Length(Vector2 v) // Calculate vector length
float Vector2LengthSqr(Vector2 v) // Calculate vector square length
float Vector2DotProduct(Vector2 v1, Vector2 v2) // Calculate two vectors dot product
float Vector2CrossProduct(Vector2 v1, Vector2 v2) // Calculate two vectors cross product
float Vector2Distance(Vector2 v1, Vector2 v2) // Calculate distance between two vectors
float Vector2DistanceSqr(Vector2 v1, Vector2 v2) // Calculate square distance between two vectors
float Vector2Angle(Vector2 v1, Vector2 v2) // Calculate the signed angle from v1 to v2, relative to the origin (0, 0) NOTE: Coordinate system
float Vector2LineAngle(Vector2 start, Vector2 end) // Calculate angle defined by a two vectors line NOTE: Parameters need to be normalized Current imp
Vector2 Vector2Scale(Vector2 v, float scale) // Scale vector (multiply by value)
Vector2 Vector2Multiply(Vector2 v1, Vector2 v2) // Multiply vector by vector
Vector2 Vector2Negate(Vector2 v) // Negate vector
Vector2 Vector2Divide(Vector2 v1, Vector2 v2) // Divide vector by vector
Vector2 Vector2Normalize(Vector2 v) // Normalize provided vector
Vector2 Vector2Transform(Vector2 v, Matrix mat) // Transforms a Vector2 by a given Matrix
Vector2 Vector2Lerp(Vector2 v1, Vector2 v2, float amount) // Calculate linear interpolation between two vectors
Vector2 Vector2Reflect(Vector2 v, Vector2 normal) // Calculate reflected vector to normal
Vector2 Vector2Min(Vector2 v1, Vector2 v2) // Get min value for each pair of components
Vector2 Vector2Max(Vector2 v1, Vector2 v2) // Get max value for each pair of components
Vector2 Vector2Rotate(Vector2 v, float angle) // Rotate vector by angle
Vector2 Vector2MoveTowards(Vector2 v, Vector2 target, float maxDistance) // Move Vector towards target
Vector2 Vector2Invert(Vector2 v) // Invert the given vector
Vector2 Vector2Clamp(Vector2 v, Vector2 min, Vector2 max) // Clamp the components of the vector between min and max values specified by the given vectors
Vector2 Vector2ClampValue(Vector2 v, float min, float max) // Clamp the magnitude of the vector between two min and max values
int Vector2Equals(Vector2 p, Vector2 q) // Check whether two given vectors are almost equal
Vector2 Vector2Refract(Vector2 v, Vector2 n, float r) // Compute the direction of a refracted ray v: normalized direction of the incoming ray n: normaliz

// Vector3 math
Vector3 Vector3Zero(void) // Vector with components value 0.0f
Vector3 Vector3One(void) // Vector with components value 1.0f
Vector3 Vector3Add(Vector3 v1, Vector3 v2) // Add two vectors
Vector3 Vector3AddValue(Vector3 v, float add) // Add vector and float value
Vector3 Vector3Subtract(Vector3 v1, Vector3 v2) // Subtract two vectors
Vector3 Vector3SubtractValue(Vector3 v, float sub) // Subtract vector by float value
Vector3 Vector3Scale(Vector3 v, float scalar) // Multiply vector by scalar
Vector3 Vector3Multiply(Vector3 v1, Vector3 v2) // Multiply vector by vector
Vector3 Vector3CrossProduct(Vector3 v1, Vector3 v2) // Calculate two vectors cross product
Vector3 Vector3Perpendicular(Vector3 v) // Calculate one vector perpendicular vector
float Vector3Length(const Vector3 v) // Calculate vector length
float Vector3LengthSqr(const Vector3 v) // Calculate vector square length
float Vector3DotProduct(Vector3 v1, Vector3 v2) // Calculate two vectors dot product
float Vector3Distance(Vector3 v1, Vector3 v2) // Calculate distance between two vectors
```

```

float Vector3DistanceSqr(Vector3 v1, Vector3 v2)
float Vector3Angle(Vector3 v1, Vector3 v2)
Vector3 Vector3Negate(Vector3 v)
Vector3 Vector3Divide(Vector3 v1, Vector3 v2)
Vector3 Vector3Normalize(Vector3 v)
Vector3 Vector3Project(Vector3 v1, Vector3 v2)
Vector3 Vector3Reject(Vector3 v1, Vector3 v2)
void Vector3OrthoNormalize(Vector3 *v1, Vector3 *v2)
Vector3 Vector3Transform(Vector3 v, Matrix mat)
Vector3 Vector3RotateByQuaternion(Vector3 v, Quaternion q)
Vector3 Vector3RotateByAxisAngle(Vector3 v, Vector3 axis, float angle)
Vector3 Vector3MoveTowards(Vector3 v, Vector3 target, float maxDistance)
Vector3 Vector3Lerp(Vector3 v1, Vector3 v2, float amount)
Vector3 Vector3CubicHermite(Vector3 v1, Vector3 tangent1, Vector3 v2, Vector3 tangent2, float amount) // Calculate cubic hermite interpolation between two vectors and their ta
Vector3 Vector3Reflect(Vector3 v, Vector3 normal)
Vector3 Vector3Min(Vector3 v1, Vector3 v2)
Vector3 Vector3Max(Vector3 v1, Vector3 v2)
Vector3 Vector3Barycenter(Vector3 p, Vector3 a, Vector3 b, Vector3 c)
Vector3 Vector3Unproject(Vector3 source, Matrix projection, Matrix view)
float3 Vector3ToFloatV(Vector3 v)
Vector3 Vector3Invert(Vector3 v)
Vector3 Vector3Clamp(Vector3 v, Vector3 min, Vector3 max)
Vector3 Vector3ClampValue(Vector3 v, float min, float max)
int Vector3Equals(Vector3 p, Vector3 q)
Vector3 Vector3Refract(Vector3 v, Vector3 n, float r)

// Vector4 math
Vector4 Vector4Zero(void)
Vector4 Vector4One(void)
Vector4 Vector4Add(Vector4 v1, Vector4 v2)
Vector4 Vector4AddValue(Vector4 v, float add)
Vector4 Vector4Subtract(Vector4 v1, Vector4 v2)
Vector4 Vector4SubtractValue(Vector4 v, float add)
float Vector4Length(Vector4 v)
float Vector4LengthSqr(Vector4 v)
float Vector4DotProduct(Vector4 v1, Vector4 v2)
float Vector4Distance(Vector4 v1, Vector4 v2)
float Vector4DistanceSqr(Vector4 v1, Vector4 v2)
Vector4 Vector4Scale(Vector4 v, float scale)
Vector4 Vector4Multiply(Vector4 v1, Vector4 v2)
Vector4 Vector4Negate(Vector4 v)
Vector4 Vector4Divide(Vector4 v1, Vector4 v2)
Vector4 Vector4Normalize(Vector4 v)
Vector4 Vector4Min(Vector4 v1, Vector4 v2)
Vector4 Vector4Max(Vector4 v1, Vector4 v2)
Vector4 Vector4Lerp(Vector4 v1, Vector4 v2, float amount)
Vector4 Vector4MoveTowards(Vector4 v, Vector4 target, float maxDistance)
Vector4 Vector4Invert(Vector4 v)
int Vector4Equals(Vector4 p, Vector4 q)

// Matrix math
float MatrixDeterminant(Matrix mat)
float MatrixTrace(Matrix mat)
Matrix MatrixTranspose(Matrix mat)
Matrix MatrixInvert(Matrix mat)
Matrix MatrixIdentity(void)
Matrix MatrixAdd(Matrix left, Matrix right)
Matrix MatrixSubtract(Matrix left, Matrix right)
Matrix MatrixMultiply(Matrix left, Matrix right)
Matrix MatrixMultiplyValue(Matrix left, float value)
Matrix MatrixTranslate(float x, float y, float z)
Matrix MatrixRotate(Vector3 axis, float angle)
Matrix MatrixRotateX(float angle)
Matrix MatrixRotateY(float angle)
Matrix MatrixRotateZ(float angle)
Matrix MatrixRotateXYZ(Vector3 angle)
Matrix MatrixRotateZYX(Vector3 angle)
Matrix MatrixScale(float x, float y, float z)
Matrix MatrixFrustum(double left, double right, double bottom, double top, double nearPlane, double farPlane) // Get perspective projection matrix
Matrix MatrixPerspective(double fovY, double aspect, double nearPlane, double farPlane) // Get perspective projection matrix NOTE: Fovy angle must be provided in radians
Matrix MatrixOrtho(double left, double right, double bottom, double top, double nearPlane, double farPlane) // Get orthographic projection matrix

// Calculate square distance between two vectors
// Calculate angle between two vectors
// Negate provided vector (invert direction)
// Divide vector by vector
// Normalize provided vector
// Calculate the projection of the vector v1 on to v2
// Calculate the rejection of the vector v1 on to v2
// Orthonormalize provided vectors Makes vectors normalized and orthogonal to each other Gram-Schmidt
// Transforms a Vector3 by a given Matrix
// Transform a vector by quaternion rotation
// Rotates a vector around an axis
// Move Vector towards target
// Calculate linear interpolation between two vectors
// Calculate cubic hermite interpolation between two vectors and their ta
// Calculate reflected vector to normal
// Get min value for each pair of components
// Get max value for each pair of components
// Compute barycenter coordinates (u, v, w) for point p with respect to triangle (a, b, c) NOTE: As
// Projects a Vector3 from screen space into object space NOTE: Self-contained function, no other r
// Get Vector3 as float array
// Invert the given vector
// Clamp the components of the vector between min and max values specified by the given vectors
// Clamp the magnitude of the vector between two values
// Check whether two given vectors are almost equal
// Compute the direction of a refracted ray v: normalized direction of the incoming ray n: normaliz

// Get vector zero
// Get vector one
// Add two vectors
// Add value to vector components
// Subtract vectors
// Subtract value from vector components
// Vector length
// Vector square length
// Vectors dot product
// Calculate distance between two vectors
// Calculate square distance between two vectors
// Scale vector components by value (multiply)
// Multiply vector by vector
// Negate vector
// Divide vector by vector
// Normalize provided vector
// Get min value for each pair of components
// Get max value for each pair of components
// Calculate linear interpolation between two vectors
// Move Vector towards target
// Invert the given vector
// Check whether two given vectors are almost equal

// Compute matrix determinant
// Get the trace of the matrix (sum of the values along the diagonal)
// Transposes provided matrix
// Invert provided matrix
// Get identity matrix
// Add two matrices
// Subtract two matrices (left - right)
// Get two matrix multiplication NOTE: When multiplying matrices... the order matters!
// Multiply matrix components by value
// Get translation matrix
// Create rotation matrix from axis and angle NOTE: Angle should be provided in radians
// Get x-rotation matrix NOTE: Angle must be provided in radians
// Get y-rotation matrix NOTE: Angle must be provided in radians
// Get z-rotation matrix NOTE: Angle must be provided in radians
// Get xyz-rotation matrix NOTE: Angle must be provided in radians
// Get zyx-rotation matrix NOTE: Angle must be provided in radians
// Get scaling matrix

```

```
Matrix MatrixLookAt(Vector3 eye, Vector3 target, Vector3 up)
float16 MatrixToFloatV(Matrix mat)
```

```
// Quaternion math
```

```
Quaternion QuaternionAdd(Quaternion q1, Quaternion q2)
Quaternion QuaternionAddValue(Quaternion q, float add)
Quaternion QuaternionSubtract(Quaternion q1, Quaternion q2)
Quaternion QuaternionSubtractValue(Quaternion q, float sub)
Quaternion QuaternionIdentity(void)
float QuaternionLength(Quaternion q)
Quaternion QuaternionNormalize(Quaternion q)
Quaternion QuaternionInvert(Quaternion q)
Quaternion QuaternionMultiply(Quaternion q1, Quaternion q2)
Quaternion QuaternionScale(Quaternion q, float mul)
Quaternion QuaternionDivide(Quaternion q1, Quaternion q2)
Quaternion QuaternionLerp(Quaternion q1, Quaternion q2, float amount)
Quaternion QuaternionNlerp(Quaternion q1, Quaternion q2, float amount)
Quaternion QuaternionSlerp(Quaternion q1, Quaternion q2, float amount)
Quaternion QuaternionCubicHermiteSpline(Quaternion q1, Quaternion outTangent1, Quaternion q2, Quaternion inTangent2, float t) // Calculate quaternion cubic spline interpolation
Quaternion QuaternionFromVector3ToVector3(Vector3 from, Vector3 to) // Calculate quaternion based on the rotation from one vector to another
Quaternion QuaternionFromMatrix(Matrix mat) // Get a quaternion for a given rotation matrix
Matrix QuaternionToMatrix(Quaternion q) // Get a matrix for a given quaternion
Quaternion QuaternionFromAxisAngle(Vector3 axis, float angle) // Get rotation quaternion for an angle and axis NOTE: Angle must be provided in radians
void QuaternionToAxisAngle(Quaternion q, Vector3 *outAxis, float *outAngle) // Get the rotation angle and axis for a given quaternion
Quaternion QuaternionFromEuler(float pitch, float yaw, float roll) // Get the quaternion equivalent to Euler angles NOTE: Rotation order is ZYX
Vector3 QuaternionToEuler(Quaternion q) // Get the Euler angles equivalent to quaternion (roll, pitch, yaw) NOTE: Angles are returned in a
Quaternion QuaternionTransform(Quaternion q, Matrix mat) // Transform a quaternion given a transformation matrix
int QuaternionEquals(Quaternion p, Quaternion q) // Check whether two given quaternions are almost equal
Matrix MatrixCompose(Vector3 translation, Quaternion rotation, Vector3 scale) // Compose a transformation matrix from rotational, translational and scaling components TODO: Th
void MatrixDecompose(Matrix mat, Vector3 *translation, Quaternion *rotation, Vector3 *scale) // Decompose a transformation matrix into its rotational, translational and scaling
```

```
// Get camera look-at matrix (view matrix)
// Get float array of matrix data
```

```
// Add two quaternions
// Add quaternion and float value
// Subtract two quaternions
// Subtract quaternion and float value
// Get identity quaternion
// Computes the length of a quaternion
// Normalize provided quaternion
// Invert provided quaternion
// Calculate two quaternion multiplication
// Scale quaternion by float value
// Divide two quaternions
// Calculate linear interpolation between two quaternions
// Calculate slerp-optimized interpolation between two quaternions
// Calculates spherical linear interpolation between two quaternions
// Calculate quaternion cubic spline interpolation
// Calculate quaternion based on the rotation from one vector to another
// Get a quaternion for a given rotation matrix
// Get a matrix for a given quaternion
// Get rotation quaternion for an angle and axis NOTE: Angle must be provided in radians
// Get the rotation angle and axis for a given quaternion
// Get the quaternion equivalent to Euler angles NOTE: Rotation order is ZYX
// Get the Euler angles equivalent to quaternion (roll, pitch, yaw) NOTE: Angles are returned in a
// Transform a quaternion given a transformation matrix
// Check whether two given quaternions are almost equal
// Compose a transformation matrix from rotational, translational and scaling components TODO: Th
// Decompose a transformation matrix into its rotational, translational and scaling
```

Other cheatsheets

- [raylib cheatsheet](#)

raylib quick reference card - Copyright (c) 2013-2026 Ramon Santamaria (@raysan5)